Speech Command Recognition with Reinforcement Learning-Based Threshold Tuning

Kean Joshua N. Antonio
Holy Angel University
Lot 205 D1-J Anusension Street, Mabiga, Mabalacat City, Pampanga
(+63) 9957305224
antoniokeanjoshua@gmail.com

Josh Adriane G. Lim
Holy Angel University
Mirus Res,
Mabiga, Mabalacat City, Pampanga
(+63) 9267801125
jglimhau@gmail.com

Glen Garcia Pasamonte
Holy Angel University
660 San Angelo Street, Lourdes Sur,
Angeles City Pampanga
(+63) 9761509739
glenpasamonte28@gmail.com

Amiel Miguel T. Vitug
Holy Angel University
36-20 Mt. Taal St. Talimundok, Dau,
Mabalacat City, Pampanga
(+63) 9662531657
atvitug1@gmail.com

1. PROBLEM STATEMENT

Voice User Interfaces (VUIs), particularly those used in IoT devices such as smart assistants, depend on speech command recognition to interpret user requests. Despite their usefulness, These systems are often affected by real-world factors such as background noise, accent variation, and differences in speaking style, which can reduce recognition accuracy. Misclassification of spoken commands may result in incorrect system responses, lower usability, and reduced user satisfaction. We propose the development of a Speech Command Recognition system that uses a Convolutional Neural Network (CNN) with spectrogram-based input features to classify short spoken commands. To further improve decision-making, Reinforcement Learning (RL) will be used to tune the classification threshold so that false positives and false negatives can be balanced according to asymmetric error costs.

2. SCOPE

The scope of this project will include audio command classification using a spectrogram-based CNN, reinforcement learning-based threshold tuning to optimize decision-making, evaluation using a public speech command dataset, and the development of an offline experimental prototype solely for research and academic purposes.

3. SUCCESS METRICS & CONSTRAINTS

The project will measure the practical performance of both the CNN-based classifier and the RL-based threshold optimization component. The CNN model is expected to achieve a minimum of 90% accuracy on the test set and a macro-F1 score of at least 0.88, ensuring credible and balanced classification across all speech command classes. Meanwhile, the RL component should achieve at least a 20% reduction in expected error cost relative to a fixed-threshold baseline, indicating a meaningful improvement in classification decision quality. As a practical constraint, the complete training and optimization process must remain computationally feasible and should finish within 90 minutes using an IDE and a single GPU with 4GB VRAM.

4. RELATED WORK

The Google Speech Commands dataset is a large collection for limited-vocabulary speech recognition, providing thousands of labeled audio recordings suitable for training deep learning models [1]. Convolutional Neural Networks have been applied to speech command recognition and improved through a hyperparameter selection method based on the Differential Evolution algorithm. The DE-based method achieved 91.5% accuracy, outperforming the Genetic Algorithm-based approach, which reached 87.7% [2]. More recent studies have shown that Mel-spectrogram-based CNN models remain effective for speech command recognition. A four-layer CNN trained on Mel spectrograms achieved 96.05% testing accuracy, with 95% micro-averaged and macro-averaged precision, recall, and F1-scores, demonstrating strong and balanced classification across command classes [3]. Transfer learning has also been applied to speech command detection, where an improved pretrained YAMNet model achieved 95.28% accuracy on the Speech Commands dataset, showing that pretrained audio models can also perform well in this task [4]. Reinforcement learning

has also been used in imbalanced classification, where a deep RL framework achieved more balanced performance than conventional methods and highlighted the importance of decision-threshold selection in classification outcomes [5]. Threshold estimation has also been studied in keyword spotting, where robust threshold selection was identified as a non-trivial task, and a score-calibration approach was proposed to simplify threshold selection and improve performance in noisy few-shot keyword spotting settings [6].

5. DATASET PLAN

The project uses a cleaned version of the Google Speech Commands Dataset v0.02, which is restored through KaggleHub using data/get_data.py. The current dataset manifest contains 105,829 one-second audio clips recorded at 16 kHz across 35 classes. Based on the current manifest, 89,080 clips were kept as usable samples, while 16,749 clips were removed during filtering. These clips are then converted into normalized log-Mel, delta, and delta-delta spectrogram features for the CNN model. For data splitting, the project follows the dataset's provided validation and testing lists, while the remaining usable clips are assigned to the training set.

6. PLANNED CNN/NLP/RL COMPONENTS AND MVP

The implemented system uses a Convolutional Neural Network to classify simple spoken commands from normalized log-Mel, delta, and delta-delta spectrogram features. To make the system more reliable, the Reinforcement Learning component acts as a Q-learning threshold-tuning agent that adjusts the classifier's confidence threshold to reduce expected cost when different errors have different penalties. In its minimum viable form, the project combines spectrogram-based CNN classification with RL-based decision-threshold optimization.

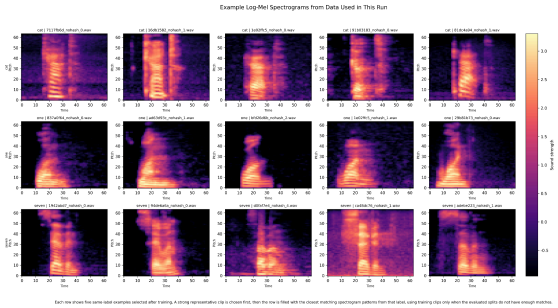
7. ETHICS STATEMENT

The Speech Command Recognition system carries three key ethical risks. First, accent and dialect bias arise because the Google Speech Commands v0.02 dataset skews toward American English speakers, potentially reducing accuracy for non-native speakers with different accents; this is mitigated through audio augmentation and per-class performance documentation. Second, there is also a privacy concern because voice data can be sensitive. However, this project only uses a public dataset licensed under CC BY 4.0, does not collect new personal audio, and does not store additional participant recordings in the repository. Third, command misinterpretation, where commands like "no" may be classified as "go" is mitigated by RL-based threshold tuning and fallback handling for low-confidence predictions. No PII (Personal Identifiable Information) is present in the dataset; all speakers are fully anonymized with random IDs. This system is only an academic prototype and is not intended for public deployment or high-stakes use.

8. RESULTS

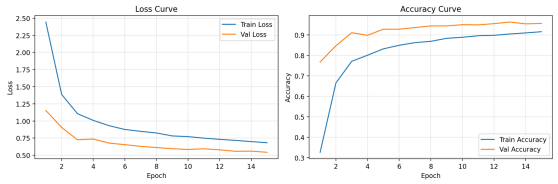
I. Main Output

Sample Log-Mel Spectrograms (medium_end)



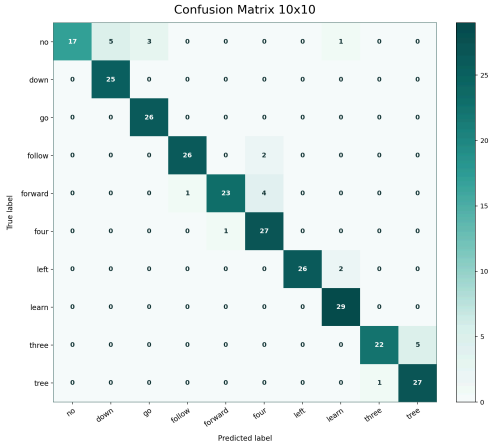
- Side-by-side examples of labeled Log-Mel spectrograms from the audio samples cat, one, and seven

Training and Validation Accuracy/Loss Curves (medium_end)



- Training and validation loss and accuracy curves for the medium_end run across 15 epochs, showing steady convergence, improving accuracy, and reduced loss as training progressed.

Confusion Matrix (medium_end)



- In this 10x10 confusion matrix, most predictions

remain on the diagonal, but some of the most noticeable misclassifications appear between forward and four, three and tree, and no and nine, showing where the model still confuses similar-sounding commands.

Final Metrics Summary (medium_end)

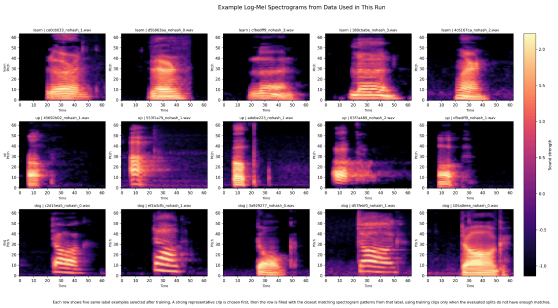
Metric	Value
Preset	medium_end
Validation Accuracy	96.30%
Validation Macro-F1	96.31%
Test Accuracy	93.00%
Test Macro-F1	92.91%
Number of Classes	35
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.60
Validation Expected Cost (Baseline)	0.185
Validation Expected Cost (Tuned)	0.141
Validation Cost Reduction	0.044
Test Expected Cost (Baseline)	0.350
Test Expected Cost (Tuned)	0.220
Test Cost Reduction	0.130

- Final metrics for the medium_end run, showing 93.00% test accuracy, 92.91% test Macro-F1, and a reduction in expected test cost from 0.35 to 0.22 after threshold tuning.

II. Ablation 1 (medium_end with no_augment)

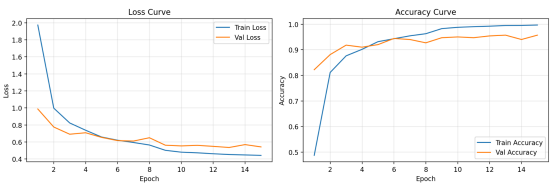
The no_augment ablation removes data augmentation during training to show how the model performs when it learns only from the original audio samples without added noise, shifting, or masking.

Sample Log-Mel Spectrograms (medium_end with no_augment)



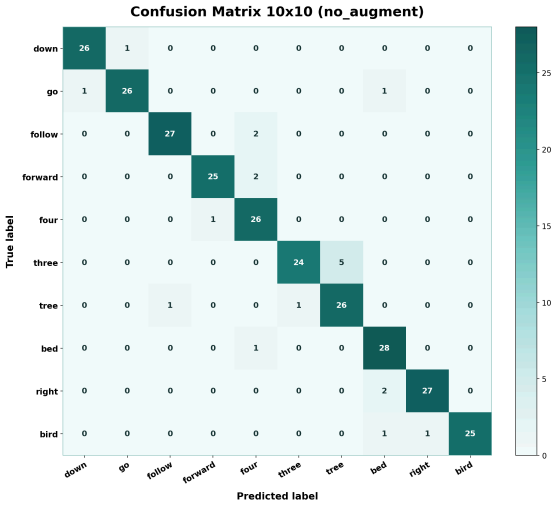
- Side-by-side examples of labeled Log-Mel spectrograms from the audio samples learn, up, dog.

Training and Validation Accuracy/Loss Curves (medium_end with no_augment)



- Training and validation curves for the no_augment run, showing steady learning across 15 epochs, with decreasing loss and improving accuracy even without data augmentation.

Confusion Matrix 10x10 (medium_end with no_augment)



- This confusion matrix from the no_augment run shows that most predictions are still correct, but the model continues to confuse several similar-sounding words. The most noticeable misclassifications appear between three and tree, forward and four, and, to a lesser extent, follow and four, while labels such as bed, right, and bird were recognized more consistently.

Final Metrics Summary (medium_end with no_augment)

Metric	Value
Preset	medium_end
Ablation	no_augment
Validation Accuracy	95.70%
Validation Macro-F1	95.72%
Test Accuracy	94.70%
Test Macro-F1	94.73%
Number of Classes	35
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.65
Validation Expected Cost (Baseline)	0.215
Validation Expected Cost (Tuned)	0.165
Validation Cost Reduction	0.050
Test Expected Cost (Baseline)	0.265
Test Expected Cost (Tuned)	0.147
Test Cost Reduction	0.118

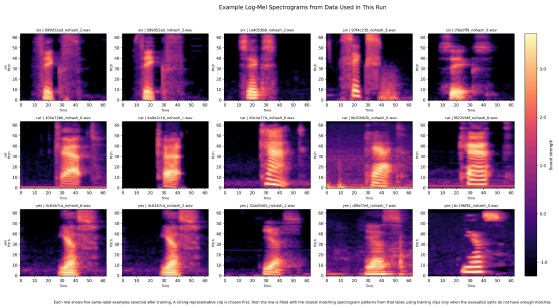
Final metrics for the no_augment run, showing that the model still performed strongly without data augmentation, with 94.70% test accuracy, 94.73% test Macro-F1, and a lower expected test cost from 0.265 to 0.147 after threshold tuning.

III. Ablation 2 (medium_end + no_balance)

The no_balance run removes the class-balancing step during training to show how

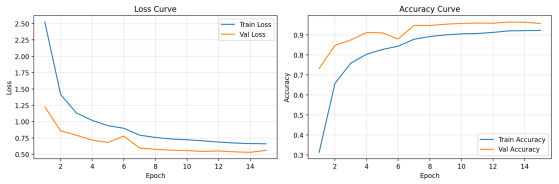
the model performs when it learns from the data without extra balancing for class distribution.

Sample Log-Mel Spectrograms (medium_end with no_balance)



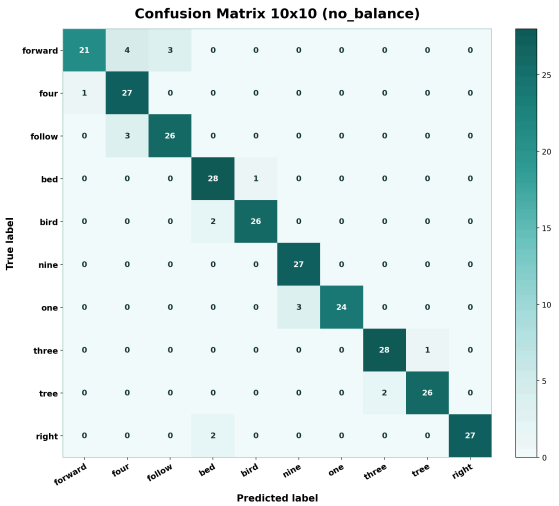
- Example Log-Mel spectrograms from the medium_end run, showing sample clips for the words left, two, and sheila and the variation in their time-frequency patterns across different recordings.

Training and Validation Accuracy/Loss Curves (medium_end with no_balance)



- Training and validation curves for the no_balance run, showing that the model still learned steadily across 15 epochs, with loss going down and accuracy improving even without class balancing.

Confusion Matrix 10x10 (medium_end with no_balance)



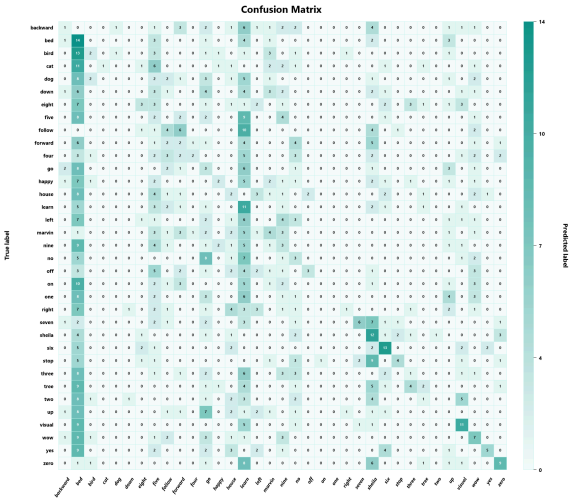
- This confusion matrix from the no_balance run shows that most predictions are still correct, but the model continues to confuse similar words such as forward, four, and follow. It also shows smaller but noticeable mix-ups between bed and bird, one and nine, and three and tree, while labels like right were recognized more consistently.

Final Metrics Summary (medium_end with no_balance)

Metric	Value
Preset	medium_end
Ablation	no_balance
Validation Accuracy	96.50%
Validation Macro-F1	96.49%
Test Accuracy	94.20%
Test Macro-F1	94.20%
Number of Classes	35
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.50
Validation Expected Cost (Baseline)	0.175
Validation Expected Cost (Tuned)	0.122
Validation Cost Reduction	0.053
Test Expected Cost (Baseline)	0.290
Test Expected Cost (Tuned)	0.235
Test Cost Reduction	0.055

- Final metrics for the no_balance run, showing 94.20% test accuracy, 94.20% test Macro-F1, and a reduction in expected test cost from 0.290 to 0.235, for a total test cost reduction of 0.055.

Non-DL Baseline Confusion Matrix (medium_end)



- The non-deep-learning baseline produced a much noisier confusion matrix with many off-diagonal errors, showing that a simple centroid-based classifier could not separate the command classes well. This highlights why the CNN was needed for the final system.

The earlier parts of this report show what was first proposed for the project. The next section presents what was actually implemented, the updated experimental setup, and the final results of the completed system.

9. FINAL REPORT

- **Problem and Motivation, Success Metrics, and Constraints**

This project aims to make speech command recognition more reliable for Voice User Interfaces (VUIs), especially in real-world situations where performance can drop because of background noise, accent differences, and variations in the way people speak. To measure how well the system performs, the project uses accuracy, Macro-F1, and expected cost after threshold tuning, since these give a better picture of both classification quality and decision reliability. In the official medium_end run, the model achieved 93.00% test accuracy and 92.91% test Macro-F1, while RL-based threshold tuning reduced the expected test cost from 0.350 to 0.220. The main challenges in the project were handling noisy and varied speech recordings while keeping training practical within limited hardware resources.

- **Dataset Used (Source, License, Collection Notes), Preprocessing, and Splits**

The project uses the Google Speech Commands Dataset v0.02, which contains 105,829 one-second audio recordings sampled at 16 kHz across 35 command classes. Based on the dataset manifest used in the current project, 89,080 clips were kept as usable samples, while 16,749 clips were removed during filtering. For the official medium_end experiment, the project used a smaller controlled subset made up of 10,000 training samples, 1,000 validation samples, and 1,000 testing samples. This made the run more practical while still giving reliable results.

Before training, the audio clips were organized through a dataset manifest and converted into normalized log-Mel, delta, and delta-delta spectrogram features. For the data split, the project followed the dataset's provided validation and testing lists, while the remaining usable clips were placed in the training set. This helped keep the evaluation consistent and reduced the risk of data leakage between training and testing.

- **Training details: hyperparameters, schedules, compute used, reproducibility steps.**

The project used the medium_end preset, which trains on a smaller and more manageable subset of the Google Speech Commands dataset instead of running on the full dataset at once. In this setup, the experiment used 10,000 training samples, 1,000 validation samples, and 1,000 testing samples. This made the training process more practical within the available hardware limits while still producing strong and reliable results.

The model was trained for 15 epochs with a batch size of 16, a learning rate of 0.001, weight decay of 0.0001, and label smoothing of 0.05. To make the model more robust, the training pipeline applied augmentation techniques such as time shifting, additive noise, background-noise mixing, gain adjustment, and SpecAugment-style masking. The main run also used balanced sampling and class-weighted loss to help reduce the effect of class imbalance. For optimization, the project used the AdamW optimizer together with a ReduceLROnPlateau learning-rate scheduler.

For computation, the run used automatic device selection, 0 DataLoader workers, feature caching, and AMP disabled for stability. To support reproducibility, the experiment kept a fixed seed of 2518392709, along with the saved dataset manifest, saved run configuration, and the one-command workflow used to reproduce the medium_end pipeline. `Python .\run.py --preset medium_end --no-open-files`.

- **Modeling: architecture(s), why chosen, CNN/NLP/RL components, and how they connect.**

The project uses a hybrid setup made up of a CNN classifier and an RL-based threshold-tuning module. There is no NLP component in this system because the task is focused on recognizing short spoken commands directly from audio, not from text. Each 1-second audio clip is first converted into normalized log-Mel, delta, and delta-delta spectrogram features. These feature maps are then passed into a residual squeeze-excitation CNN with a convolutional stem, gradually increasing channels, adaptive average pooling, and a final classifier head.

CNN was chosen because spectrograms are two-dimensional time-frequency representations, and CNNs are well suited for learning patterns from this kind of input. The RL component was added because the project is not only about predicting the correct class, but also about making better decisions when the model is uncertain. Instead of always accepting the top prediction, the system uses threshold tuning to control when a prediction is confident enough to keep.

After the CNN produces class probabilities, those confidence scores are passed to a Q-learning RL agent, which searches for the best decision threshold on the validation set. That threshold is then applied during the final decision stage so that low-confidence predictions can be rejected when needed. In this way, the CNN handles feature learning and classification, while the RL component

improves decision reliability by adjusting the confidence threshold in a cost-sensitive way.

• Evaluation: metrics, baselines, ablations (≥2), error/slice analysis, calibration

To fairly evaluate the system, we compared the final CNN-based model against a simple non-deep-learning baseline. For the baseline, we used a nearest-centroid classifier on the same log-Mel-based features, but its performance was much weaker, reaching only 16.00% validation accuracy and 13.30% test accuracy, with 14.68% validation Macro-F1 and 12.70% test Macro-F1. Its confusion matrix also showed many off-diagonal errors, meaning that it struggled to separate the command classes. In contrast, the CNN performed much better, achieving 96.30% validation accuracy, 96.31% validation Macro-F1, 93.00% test accuracy, and 92.91% test Macro-F1. This large gap in performance shows that a simple classical baseline was not enough for this task, which is why the CNN was used as the final model.

For the official medium_end run, RL-based threshold tuning selected a confidence threshold of 0.60. With this threshold, the model reduced the validation expected cost from 0.185 to 0.141 and the test expected cost from 0.350 to 0.220. To better understand which training components were helping the model, two ablation studies were also conducted. In the no_augment run, where data augmentation was removed, the model still achieved 94.70% test accuracy and 94.73% test Macro-F1, while reducing the test expected cost from 0.265 to 0.147 after threshold tuning. In the no_balance run, where class balancing was removed, the model achieved 94.20% test accuracy and 94.20% test Macro-F1, with the test expected cost reduced from 0.290 to 0.235. These results show that the model remained strong even when individual training components were removed, while also suggesting that the exact augmentation and balancing settings still affect final performance.

Error analysis was done using the confusion matrix. Most predictions stayed on the diagonal, which means that the model correctly classified many of the commands, but there were still some noticeable mistakes between similar-sounding words. Some of the clearer examples were forward and four, down and cat, and no and nine. This gave us useful class-level insight into where the model still struggled. However, the project did not include a separate slice analysis for speaker, accent, or demographic subgroups because those metadata were not available in the dataset. There was also no separate calibration module added. Instead, the model's confidence scores were used directly in the RL-based threshold-tuning step to improve decision quality under asymmetric error costs.

• Ethics & Policy: risks, mitigations, intended use & limitations, privacy, fairness checks.

This project is intended for limited-vocabulary speech command recognition in Voice User Interfaces and other similar low-risk applications. It is not designed for surveillance, speaker identification, or any high-stakes decision-making. The main risks of the system include misrecognized commands, weaker performance in noisy environments, and uneven performance across different accents or speaking styles. To help reduce these risks, the project uses augmentation, class-balancing methods, and RL-based threshold tuning so that low-confidence predictions can be handled more carefully instead of always being accepted.

From a privacy perspective, the project only uses a public dataset and does not collect any new personal audio. The dataset also does not include participant details such as age, gender, or location, which helps reduce privacy concerns. However, voice data is still a sensitive type of data in general, so the project is framed strictly as an academic prototype rather than a system for real-world deployment. In terms of fairness, formal auditing across demographic groups was not possible because the dataset does not provide demographic or speaker subgroup metadata. Because of that, fairness evaluation was limited to class-level metrics and confusion-matrix-based error analysis. A major limitation of the model is that it was trained only on short English command clips, which means its performance may drop for other languages, broader speech tasks, or more diverse real-world speaking conditions.

• Model Card summary with deployment guidance/disclaimers.

The project's model is a residual squeeze-excitation CNN combined with a Q-learning RL threshold-tuning module for limited-vocabulary speech command recognition. It uses the Google Speech Commands Dataset v0.02 and processes 1-second, 16 kHz audio clips by converting them into normalized log-Mel, delta, and delta-delta spectrogram features. In the official medium_end run, the model achieved 93.00% test accuracy and 92.91% test Macro-F1, while threshold tuning reduced the expected test cost from 0.350 to 0.220. The model is intended for low-risk Voice User Interface tasks such as simple command recognition and interface control, and it

should not be used for speaker identification, surveillance, or other high-stakes applications. Deployment should include a fallback or abstain mechanism for low-confidence predictions, validation in the target acoustic environment, and awareness that performance may drop under background noise, accent variation, different speaking styles, or non-English speech.

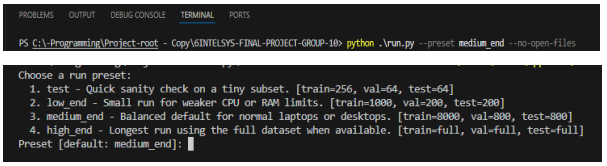
MODEL CARD

Model Card: Speech Command Recognition with CNN + RL Threshold Tuning	
Model Details	• Residual squeeze-excitation CNN classifier with RL-based threshold tuning • Uses 1-second, 16 kHz audio clips • Input features are normalized log-Mel, delta, and delta-delta spectrograms • Trained using AdamW with a ReduceLROnPlateau scheduler • Official final preset: medium_end
Intended Use	• Academic research and low-risk VUI tasks; not for surveillance, speaker identification, or high-stakes deployment
Dataset	• Google Speech Commands Dataset v0.02 (Warden, 2018), containing 105,829 total clips, 35 classes, about 2,600 speakers
Evaluation Data	• Evaluated on Google Speech Commands Dataset v0.02 • Uses the dataset's provided validation and testing lists • Current manifest contains 105,829 total clips across 35 classes • 89,080 clips were kept and 16,749 were removed during filtering
Training Data	• Uses the same cleaned Speech Commands dataset as the evaluation data • Official medium_end run uses 10,000 training, 1,000 validation, and 1,000 testing samples • Audio is converted into spectrogram-based features before training • Main setup also uses augmentation, balanced sampling, and class-weighted loss
License	Creative Commons BY 4.0
Input	1-second, 16 kHz audio converted into normalized log-Mel, delta, and delta-delta features
Metrics	• Evaluation metrics include accuracy ≥ 90%, Macro-F1 ≥ 0.88, RL cost reduction ≥ 20%
Performance	• The model achieved 93.00% test accuracy and 92.91% test Macro-F1, while RL threshold tuning reduced expected test cost from 0.350 to 0.220
Factors	• Performance can be affected by background noise • Accent variation and speaking style can change results • Microphone quality and recording conditions also matter
Limitation	• Performance may drop under noise, accent variation, non-English speech, and more diverse real-world speaking conditions.
Ethical Considerations	• Accent and dialect bias may be present, reducing the accuracy for non-native speakers • Voice data may be personal and sensitive by nature • Formal demographic fairness auditing was not possible • Fairness evaluation is limited to class-level metrics and confusion-matrix analysis
Caveats and Recommendations	• Performance may drop in noisy or more varied real-world settings • The model was trained only on short English command clips • It should not be used for high-stakes deployment • Low-confidence prediction handling should be kept in any prototype use • Future work can focus on stronger robustness and better handling of similar-sounding commands

• How to run: one-command reproduction, environment setup, and demo instructions.

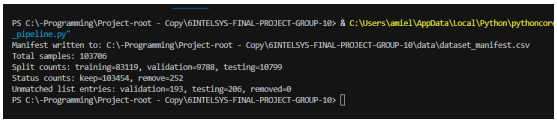
The project is designed to be reproducible through a simple setup and one-command workflow. The recommended environment uses Python 3.11. The environment can be prepared by creating the Conda environment from environment.yml, activating it, and installing the required packages from requirements.txt. If necessary, a system-specific PyTorch build should be installed first before the remaining dependencies.

For one-command reproduction, the official final configuration uses the medium_end preset. After restoring the dataset, the full pipeline can be executed with:



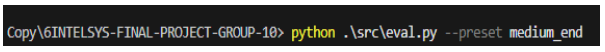

```
python .\run.py --preset medium_end
```

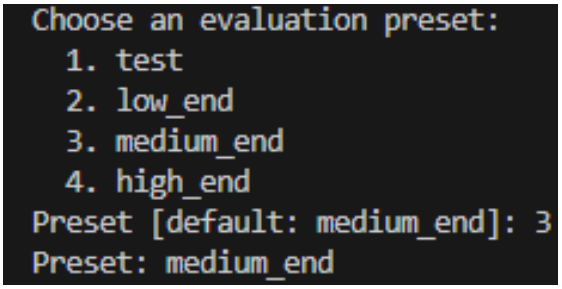
This command runs preprocessing, training, and evaluation in sequence. For demonstration, the dataset may first be checked or restored using `cd .\data` and `python .\get_data.py`, then the manifest can be built using:

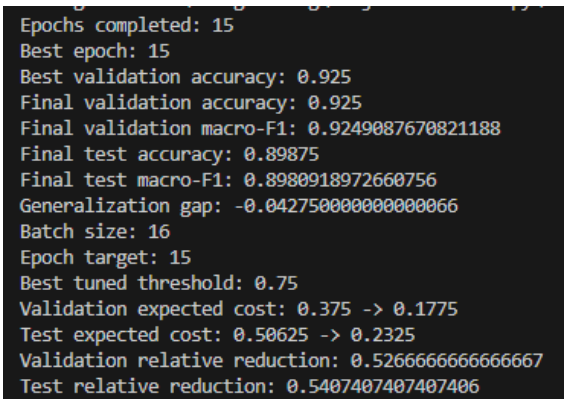



```
python .\src\data_pipeline.py
```

The trained model can then be evaluated using:








```
python .\src\eval.py --preset medium_end
```

The final metrics, confusion matrix, and other generated outputs are stored in the experiments/results/medium-end/ directory, while logs and training curves are saved in experiments/logs/medium-end/.

10. TEAM ROLES & MILESTONES

The project team is organized into four key roles to ensure efficient development and successful system integration. Specifically, the team is divided into these specific roles for the project:

- Member 1 - Kean as the Project Lead / Integration oversees the project's overall direction, coordination, and progress, ensuring that data preparation, model development, evaluation, and deployment remain aligned with the objectives and timeline. This role also manages task distribution, team communication, and the integration of outputs into a unified system.
- Member 2 - Glen would be the Data & Ethics Lead manages the dataset and ensures that all data-related processes follow ethical and legal standards. Responsibilities include sourcing and validating datasets, checking data quality and suitability, organizing preprocessing, ensuring compliance with licensing requirements, and addressing fairness, bias mitigation, privacy protection, and responsible AI practices.
- Member 3 - Amiel as the Modeling Lead focuses on designing, building, and improving the machine learning models used in the project. This includes selecting an appropriate CNN architecture for speech classification, preparing the Mel-spectrogram input pipeline, tuning model parameters, and monitoring training performance to improve accuracy, robustness, efficiency, and generalization.
- Member 4 - Josh as the Evaluation & MLOps Lead handles system testing and the operational side of the machine learning workflow. This role evaluates model performance using metrics such as accuracy, macro-F1 score, and cost reduction, documents experimental results, and compares outcomes against project goals. It also manages experiment tracking, version control, runtime

monitoring, reproducibility, and deployment practices to ensure that the system can be maintained and replicated effectively.

To maintain steady progress, the project is divided into structured development milestones.

- Week 1 Proposal & Setup focuses on finalizing the project proposal, defining the workflow, and preparing the required tools, environment, and resources.
- Week 2 Dataset Preparation, Baseline Training, and Initial CNN Results centers on dataset preparation, audio preprocessing, baseline model training, and obtaining initial CNN performance results.
- Week 3 RL Integration, Final Evaluation, and Report Preparation involves integrating the reinforcement learning component, conducting final system evaluation, and completing the project report and documentation.

11. REFERENCES

[1] Pete Warden. 2018. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. arXiv:1804.03209. Retrieved from https://arxiv.org/abs/1804.03209

[2] Sandipan Dhar, Anuvab Sen, Aritra Bandyopadhyay, Nanda Dulal Jana, Arjun Ghosh, and Zahra Sarayloo. 2023. Differential Evolution Algorithm based Hyper-Parameters Selection of Convolutional Neural Network for Speech Command Recognition. arXiv:2310.08914. Retrieved from https://arxiv.org/abs/2310.08914

[3] Dejan Vujičić, Đorđe Damnjanović, Dušan Marković, and Zoran Stamenković. 2025. Deep Learning System for Speech Command Recognition. *Electronics* 14, 19 (Sep. 2025), 3793. https://doi.org/10.3390/electronics14193793

[4] Sidahmed Lachenani, Hamza Kheddar, and Mohamed Ouldzmirli (2025). Improving pretrained YAMNET for enhanced speech command detection via transfer learning. arxiv.2504.19030. Retrieved from https://doi.org/10.48550/arxiv.2504.19030

[5] Jenny Yang, Rasheed El-Bouri, Odhran O'Donoghue, Alexander S. Lachapelle, Andrew A. S. Soltan, and David A. Clifton (2022). Deep Reinforcement Learning for Multi-class Imbalanced Training. arXiv:2205.12070. Retrieved from arXiv:2205.12070

[6] Kevin Wilkinghoff, Alessia Cornaggia-Urrigshardt, and Zheng-Hua Tan. (2025). Quantization-Based Score Calibration for Few-Shot Keyword Spotting with Dynamic Time Warping in Noisy Environments. arXiv.2510.15432 Retrieved from https://doi.org/10.48550/arXiv.2510.15432